

Subsystem B Design Document

Memory Management & Virtual Memory Simulation

Author: Amir Taylor and Fletcher Baccus

Course: COSC 514 - Operating Systems

Project: MOSS - Mini Operating System Services Simulator

1. Overview

Subsystem B simulates virtual memory management in user space. It maintains a page table mapping logical pages to physical frames, accepts a page replacement algorithm selection, processes a sequence of logical memory accesses, and reports per-access results along with aggregate statistics including page faults, hits, and hit rate.

All memory management is logical - no real memory allocation or OS calls are used. Addresses are 16-bit integers, pages are 256 bytes, and physical memory is modeled as a fixed array of 4 frames.

2. Data Structures

2.1 Address Space Configuration

The memory system is defined by three compile-time constants:

```
static const int OFFSET_BITS = 8;    // bits for page offset
static const int PAGE_SIZE  = 256;   // 2^8 bytes per page
static const int NUM_FRAMES = 4;     // physical frames available
static const int MAX_ADDR   = 65535; // highest valid logical address
```

A 16-bit logical address is split into two fields:

Field	Bits	Description
Page Number	Upper 8 bits	Identifies which page (0-255)
Offset	Lower 8 bits	Byte position within the page (0-255)

2.2 Sim (struct)

The core simulator state shared by both page replacement algorithms:

Field	Type	Description
frames	vector<int>	Array of NUM_FRAMES slots; each holds a loaded page number, or -1 if empty
pageFrame	unordered_map<int,int>	Maps page number to the frame index it currently occupies
faults	int	Count of page faults encountered so far
hits	int	Count of accesses that found the page already loaded

The struct also exposes three member functions:

Function	Description
freeFrame()	Scans frames[] and returns the index of the first empty slot, or -1 if all frames are occupied
load(page, frame, evictedPage)	Places a page into a frame, optionally evicting an existing page first and removing it from pageFrame
printFrames()	Prints the current contents of all frames to stdout

3. Algorithm Design

3.1 FIFO (First-In, First-Out)

FIFO evicts the page that has been resident in memory the longest. A separate `std::vector<int>` (order) tracks insertion order, acting as a queue. On each access:

- If the page is already in a frame: record a hit, print the physical address, and return.
- If a free frame exists: load the page there and append it to order.
- If all frames are full: remove the front entry of order to identify the eviction victim, reclaim its frame, load the new page, and append it to order.

FIFO is simple and requires no timestamp tracking, but can suffer from Belady's anomaly - adding more frames can sometimes increase page faults with certain reference strings.

3.2 LRU (Least Recently Used)

LRU evicts the page whose most recent access is furthest in the past. A separate `std::unordered_map<int,int>` (`lastUsed`) maps each loaded page to the logical clock value of its last access. An integer clock increments on every access. On each access:

- If the page is already in a frame: record a hit, update `lastUsed[page] = clock`, and return.
- If a free frame exists: load the page and record `lastUsed[page] = clock`.
- If all frames are full: scan `pageFrame` to find the entry with the smallest `lastUsed` value. Evict that page, reclaim its frame, and load the new page.

LRU generally outperforms FIFO on workloads with temporal locality because it keeps recently-accessed pages resident. The trade-off is the overhead of maintaining and scanning the `lastUsed` map on every fault.

3.3 Idle / Free Frame Handling

Both algorithms handle the initial warm-up period identically. Frames start as -1 (empty). The first `NUM_FRAMES` distinct pages each trigger a fault that fills a free frame rather than evicting anything. Once all frames are occupied, the respective eviction policy takes over. This matches standard simulator behavior where cold-start faults are unavoidable.

4. Address Translation

Three inline helper functions handle all address arithmetic:

```
int pageOf(int addr)      -> addr >> OFFSET_BITS
int offsetOf(int addr)    -> addr & (PAGE_SIZE - 1)
int physical(int frame, int offset) -> (frame << OFFSET_BITS) | offset
```

On every access the logical address is split into its page number and offset. If the page is loaded, the physical address is computed as:

$$\text{physical address} = (\text{frame_number} \ll 8) \mid \text{offset}$$

This mirrors how a real MMU translates addresses, with the frame number substituted for the page number in the upper bits.

5. Statistics & Output

After all accesses are processed, `printSummary()` reports three values:

Metric	Formula	Description
Page Faults	Incremented per fault	Total accesses that required loading a page
Hits	Incremented per hit	Total accesses that found the page already resident
Hit Rate	$(\text{hits} / \text{total}) * 100\%$	Percentage of accesses served without a page fault

Output is printed directly to stdout during simulation. Each access line shows whether it was a HIT or FAULT, the page and frame numbers, and the resulting physical address in hexadecimal. The current frame state is printed after every access for step-by-step tracing.

6. Demo Mode

The built-in demo (menu option 3) runs both algorithms against the same reference string and prints results side by side for direct comparison:

Reference string: { 1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5 }

This sequence is deliberately chosen to highlight the behavioral difference between FIFO and LRU. With 4 frames, page 5's arrival forces an eviction - FIFO evicts by insertion order while LRU evicts by recency of use, leading to different hit rates on the tail of the sequence.

7. Assumptions & Limitations

- All logical addresses are non-negative integers in the range 0-65535; no fractional or negative addresses are modeled.
- Physical memory is exactly NUM_FRAMES = 4 frames; this is a compile-time constant and not configurable at runtime.
- Page size is fixed at 256 bytes (8-bit offset). Changing OFFSET_BITS requires recompilation.
- The simulator is single-threaded and fully deterministic; concurrent accesses are not modeled.
- No actual data is stored in frames; only page numbers are tracked. The simulation models address translation behavior only.
- Both algorithms are independent sessions - running FIFO then LRU starts each from an empty frame state.
- Input validation in interactive mode accepts any integer in range but does not guard against non-integer input gracefully beyond a basic cin check.